



UNIVERSIDA TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DÍGITAL
INGENIERÍA DE SOFTWARE



ASIGNATURA:

APLICACIONES WEB

TEMA:

PRACTICA EXPERIMENTAL UNIDAD I

DOCENTE:

GUERRERO ULLOA GLEISTON CICERON

INTEGRANTES:

DARWIN ORLANDO ARCALLE GREFA
ZAMORA ARIAS CARLA ESTHEFANIA
ZAMORA BUMBILA DIEGO ALEXANDER

NIVEL:

5TO SOFTWARE

PARALELO:

“B”

FECHA:

20/05/2026

Índice

1.	Introducción.....	3
2.	Desarrollo.....	3
2.1.	HTML y HTML5 para el desarrollo web	3
2.1.1.	Fundamentos de HTML	4
2.1.2.	Elementos semánticos de HTML5.	5
2.1.3.	Formularios HTML5 y validación nativa.	6
2.1.4.	Multimedia y APIs de HTML5.	7
2.1.5.	XML y familias de lenguajes de marcado.	8
2.2.	CSS para el Diseño y la Presentación Web	8
2.2.1.	Fundamentos de CSS.	9
2.2.2.	Modelo de caja y maquetación CSS.....	10
2.2.3.	Diseño Flexbox y CSS Grid.	11
2.2.4.	Diseño responsivo y mobile first.....	15
2.2.5.	CSS3 avanzado y preprocesadores.	16
2.3.	JavaScript para Aplicaciones Web.....	17
2.3.1.	Fundamentos de JavaScript.	18
2.3.2.	Estructuras de control y funciones.	18
2.3.3.	Manipulación del DOM y eventos.....	19
2.3.4.	JavaScript moderno (ES6+) y programación asíncrona.....	20
3.	Auditoria de calidad.....	22
3.1.	Auditoria con lighthouse.....	22
3.1.1.	Corrección y mejora.....	24
3.2.	Auditoria con wave.....	25
3.3.	Tabla comparativa de métricas.....	25
4.	Conclusión.....	26
5.	Referencias.....	26
6.	Anexo.....	30

Fundamentos y diseño web

1. Introducción

El desarrollo web ha evolucionado significativamente en las últimas décadas, convirtiéndose en una de las áreas más importantes dentro de la ingeniería de software y las tecnologías de la información. Actualmente, las aplicaciones web no solo permiten mostrar información estática, sino también ofrecer experiencias dinámicas, interactivas y adaptables a distintos dispositivos y contextos de uso. Este avance ha sido posible gracias a la integración de tecnologías fundamentales como HTML5, CSS3 y JavaScript, las cuales constituyen la base del desarrollo frontend moderno. Estas tecnologías trabajan de manera complementaria: HTML define la estructura del contenido, CSS controla el diseño y la presentación visual, mientras que JavaScript proporciona interactividad y comportamiento dinámico en las aplicaciones web [1], [2].

HTML5 representa una evolución importante de los lenguajes de marcado utilizados en la web, incorporando elementos semánticos, soporte multimedia nativo y APIs avanzadas que permiten desarrollar aplicaciones más eficientes y accesibles sin depender de complementos externos [3]. Por otro lado, JavaScript se ha consolidado como el lenguaje de programación principal para el desarrollo de aplicaciones web interactivas. La evolución del estándar ECMAScript (ES6+) introdujo características modernas como módulos, funciones flecha, programación asíncrona y promesas, facilitando la creación de aplicaciones más escalables, mantenibles y eficientes [4].

En conjunto, estas tecnologías forman el núcleo del desarrollo web moderno y son esenciales para la construcción de sitios y aplicaciones que cumplan con los estándares actuales de rendimiento, accesibilidad, usabilidad y compatibilidad multiplataforma. Por esta razón, el estudio de HTML5, CSS3 y JavaScript resulta muy importante para comprender las bases técnicas y metodológicas que sustentan el desarrollo de aplicaciones web contemporáneas.

2. Desarrollo

En esta sección se abordan los principales fundamentos y tecnologías utilizadas en el desarrollo web moderno. Se estudian los conceptos esenciales de HTML y HTML5 para la creación y organización del contenido web, CSS para el diseño visual y la maquetación de páginas responsivas, y JavaScript como lenguaje de programación orientado a la interactividad y funcionalidad de las aplicaciones web. Además, se analizan herramientas y características avanzadas que permiten mejorar la experiencia del usuario, optimizar el desarrollo y aplicar buenas prácticas dentro de la creación de sitios y aplicaciones web actuales.

2.1. HTML y HTML5 para el desarrollo web

HTML es el lenguaje base de las páginas web, diseñado para describir la estructura y significado del contenido en la Web [5]. Originalmente surgió como lenguaje semántico para documentos científicos, pero ha evolucionado para soportar todo tipo de contenido web. La especificación HTML5 (2014) definió la quinta revisión de este lenguaje,

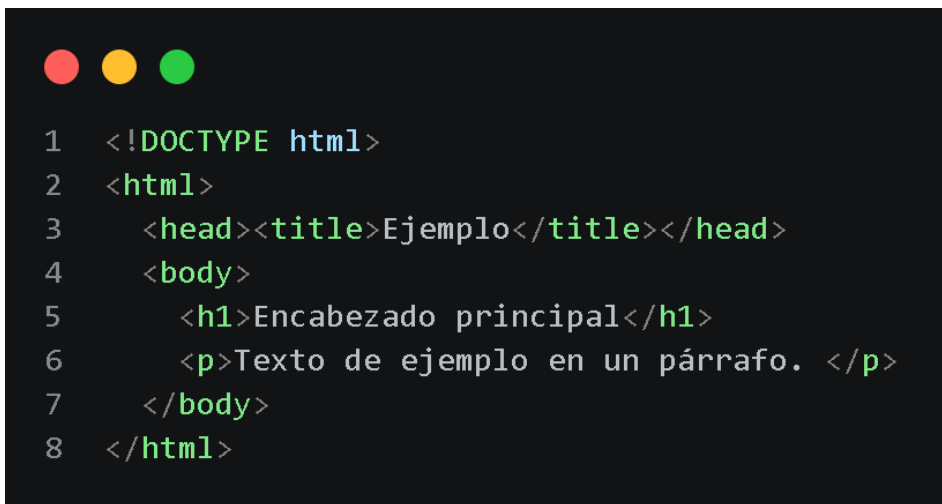
orientada a aplicaciones web interactivas, incluyendo nuevas funcionalidades y APIs para mejorar la interoperabilidad [1], [3].

En la práctica, HTML5 es la sintaxis recomendada (transmitida como text/html), compatible con navegadores modernos, mientras que XHTML5 es la variante basada en XML (transmitida como application/xhtml+xml) que exige sintaxis estricta [6].

En resumen, HTML5 nos ayuda a reforzar el rol de HTML en el desarrollo web al integrar de forma nativa elementos estructurales, multimedia y de scripting avanzados, permitiendo crear páginas más completas sin depender de tecnologías externas [1].

2.1.1. Fundamentos de HTML

Un documento HTML comienza típicamente con una declaración de tipo (`<!DOCTYPE html>`) seguida de las etiquetas básicas `<html>`, `<head>` y `<body>`. Por ejemplo:

A screenshot of a code editor with a dark background and light green text. At the top left, there are three colored circles: red, yellow, and green. The code is as follows:

```
1 <!DOCTYPE html>
2 <html>
3   <head><title>Ejemplo</title></head>
4   <body>
5     <h1>Encabezado principal</h1>
6     <p>Texto de ejemplo en un párrafo. </p>
7   </body>
8 </html>
```

Fig. 1. Estructura general de HTML

En este ejemplo, `<h1>` y `<p>` son elementos de HTML. Cada elemento HTML se escribe entre `<etiqueta>` y `</etiqueta>` (tags de apertura y cierre) y puede incluir atributos para aportar información adicional. Por ejemplo, `` incluye los atributos `src` y `alt`. El HTML forma un árbol de nodos (DOM) donde los elementos se anidan unos dentro de otros. Las propiedades semánticas de estos elementos definen su propósito: `<h1>` indica un encabezado de primer nivel, `<p>` un párrafo, `<ul>` una lista, etc. Es buena práctica usar una estructura entendible y declarar el DOCTYPE para asegurar compatibilidad entre navegadores [5] [6].

Ventajas y buenas prácticas: HTML es simple y altamente compatible. Al mantener una jerarquía lógica de etiquetas (encabezados en orden, listas para conjuntos, etc.) se mejora la accesibilidad y la mantenibilidad del código. Se recomienda usar comentarios `<!-- ... -->` con moderación y emplear nombres de id/class descriptivos. El DOCTYPE actual (`<!DOCTYPE html>`) obliga a los navegadores a interpretar el documento con las reglas modernas de HTML5 [6]. Aunque HTML permite cierto código erróneo (lo

ignora), es mejor validar el marcado regularmente (con herramientas W3C) para asegurar un documento bien formado [6].

2.1.2. Elementos semánticos de HTML5.

Los elementos semánticos definen el significado del contenido. A diferencia de `<div>` o `<span>` (genéricos), las etiquetas semánticas describen su propósito: `<header>`, `<nav>`, `<article>`, `<section>`, `<aside>`, `<footer>`, `<main>`, etc.



¿Qué es el HTML Semántico?

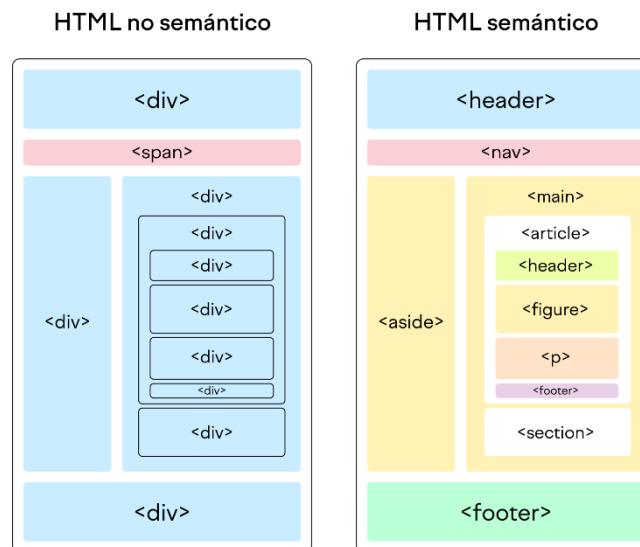


Fig. 2. Estructura semántica [7].

Por ejemplo, `<article>` encierra contenido independiente (como una entrada de blog), `<nav>` indica un menú de navegación, y `<aside>` contenido complementario no central [1] [7]. El uso de etiquetas semánticas mejora la lectura del código y la accesibilidad (los lectores de pantalla y buscadores las interpretan fácilmente) [1], [8].

```
1 <main>
2   <header>
3     <h1>Título principal</h1>
4     <nav>... menú de navegación ...</nav>
5   </header>
6   <article>
7     <h2>Artículo</h2>
8     <p>Contenido del artículo...</p>
9   </article>
10  <footer>Pie de página del sitio</footer>
11 </main>
12
```

Fig. 3. Estructura semántica en código HTML.

Ventajas y buenas prácticas: Emplear etiquetas semánticas mejora el SEO y la accesibilidad [1]. Se recomienda evitar usar `<div>` como contenedor principal cuando existe un elemento semántico adecuado [1]. Por ejemplo, siempre que el contenido sea un encabezado, usar `<header>`; si es contenido principal, `<main>`; si es contenido adicional, `<aside>`. Esto hace fácil la comprensión del documento tanto para desarrolladores como para máquinas[8]. Sin embargo, si no hay un elemento semántico específico, `<div>` aún es útil para agrupar contenido estructural sin significado.

2.1.3. Formularios HTML5 y validación nativa.

HTML5 introdujo nuevos tipos de entrada (type) y validaciones integradas en el propio navegador. Por ejemplo, `<input type="email">` obliga a ingresar una dirección de correo válida, `<input type="url">` valida una URL, y `<input type="number" min="0" max="10">` restringe números entre 0 y 10 [9], [10]. Además, atributos como `required`, `pattern`, `min`, `max` permiten validar sin JavaScript adicional [9][10]. Por ejemplo:

```
1 <form>
2   <label for="email">Correo: </label>
3   <input type="email" id="email" name="email" required>
4   <label for="edad">Edad: </label>
5   <input type="number" id="edad" name="edad" min="18" max="70" required>
6   <input type="submit" value="Enviar">
7 </form>
```

Fig. 4. Formulario simple en HTML.

Aquí, el navegador verifica automáticamente que el campo de correo contenga un formato válido y la edad esté en el rango especificado.

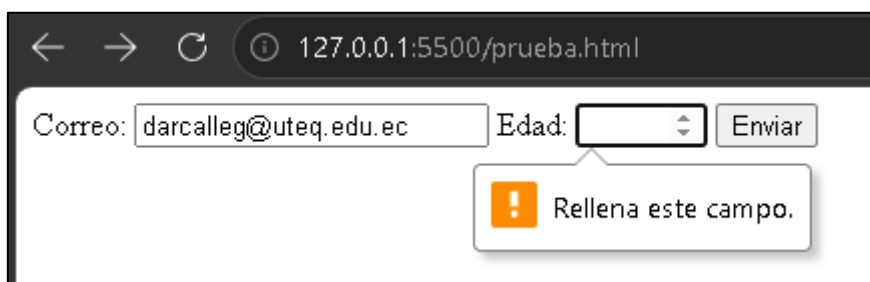


Fig. 5. Visualización de formulario simple en HTML.

Ventajas y limitaciones: La validación nativa mejora la experiencia al detectar errores sin JS [9]. Reduce código y previene errores básicos (email incorrecto, campo vacío, patrón no cumplido). Sin embargo, la apariencia de mensajes depende del navegador y puede ser limitada. Además, siempre se debe validar también en el servidor por seguridad. Los casos más complejos (por ejemplo, validaciones inter-campos) aún requieren JavaScript o el Constraint Validation API [9]. Las buenas prácticas incluyen añadir `novalidate` si se controlará la validación con JS, y usar `<label>` vinculados con `for` para mejorar la accesibilidad.

2.1.4. Multimedia y APIs de HTML5.

HTML5 facilita el manejo de multimedia en la web. Se incluyen los elementos `<video>` y `<audio>` para integrar video y audio nativos sin plugins [1]. Por ejemplo:

```
1 <video controls width="640" height="360">
2   <source src="recursos/Video_Vector_Empresas.mp4" type="video/mp4">
3   Tu navegador no soporta el elemento de video.
4 </video> <br>
5
6 <audio controls>
7   <source src="recursos/Coporative_Music.mpeg" type="audio/mpeg">
8   Tu navegador no soporta el elemento de audio.
9 </audio>
```

Fig. 6. Video y Audio en HTML.

Estos elementos soportan múltiples formatos mediante etiquetas `<source>`, ofrecen controles nativos y pueden incluir subtítulos (`<track>`).

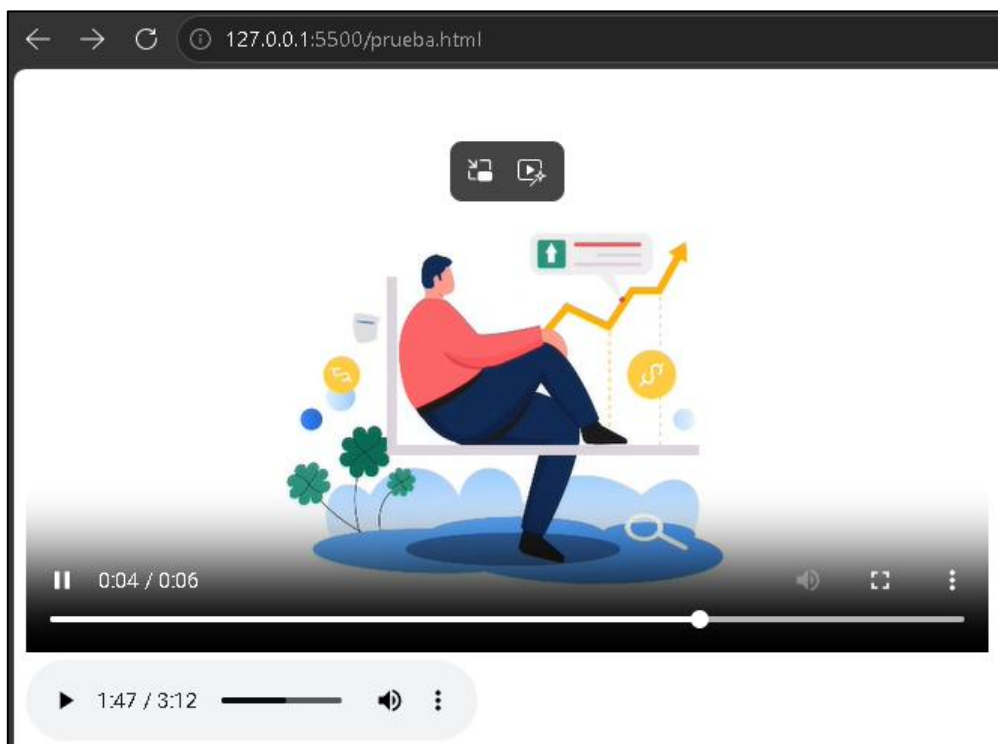


Fig. 7. Visualización de audio y video en HTML.

Para gráficos dinámicos, HTML5 introdujo `<canvas>`, que permite dibujar gráficos 2D con JavaScript. Además, las APIs de HTML5 enriquecen las páginas: por ejemplo, Web Audio API para procesamiento de sonido, WebGL para gráficos 3D en `<canvas>`, Drag-and-Drop, geolocalización, almacenamiento local (LocalStorage/IndexedDB), entre otras [1], [11].

Ventajas y limitaciones: Las etiquetas `<video>` y `<audio>` evitan dependencias de terceros y son compatibles con estándares web [1]. Se recomienda proveer múltiples

formatos (MP4, WebM, Ogg) para máxima compatibilidad y añadir subtítulos para accesibilidad. El elemento <canvas> es muy versátil para animaciones o visualización de datos, pero requiere programación con JS para usarlo. Las limitaciones incluyen diferencias de soporte de formatos entre navegadores y el peso de los archivos multimedia. Buenas prácticas incluyen proveer siempre contenido alternativo (texto o enlaces) para navegadores que no soporten los formatos, y utilizar APIs de forma progresiva (verificar disponibilidad antes de usarlas).

2.1.5. XML y familias de lenguajes de marcado.

XML (Extensible Markup Language) es un lenguaje de marcado de propósito general inspirado en HTML, pero diseñado para definir vocabularios propios[12]. A diferencia de HTML, XML no tiene etiquetas predefinidas; en su lugar el autor define etiquetas según sus necesidades. La principal función de XML es el intercambio de datos estructurados entre sistemas. Por ejemplo:

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) at the top left. The code is written in a light green font and shows an XML structure for a note. It consists of five lines: a root tag <nota>, an opening tag <para> followed by the text 'Pedro', a closing tag </para>, an opening tag <de> followed by the text 'Ana', a closing tag </de>, an opening tag <mensaje> followed by the text '¡Hola!', a closing tag </mensaje>, and finally a closing tag </nota>.

```
1 <nota>
2   <para>Pedro</para>
3   <de>Ana</de>
4   <mensaje>¡Hola!</mensaje>
5 </nota>
```

Fig. 8. Ejemplo de código XML.

Muchas tecnologías web usan lenguajes basados en XML: XHTML (HTML reformulado como XML), SVG (gráficos vectoriales), MathML (notación matemática), RSS/Atom (feeds), entre otros [12]. HTML5 define dos sintaxis: la sintaxis HTML normal (p. ej. text/html) y la sintaxis XHTML (p. ej. application/xhtml+xml), que es una aplicación de XML [3]. En XHTML (como en cualquier XML) la sintaxis es estricta: incluso un error menor (falta un cierre de etiqueta) provocará que el navegador no muestre el documento [3]. En HTML normal, por el contrario, se corrigen errores menores automáticamente.

Ventajas y usos actuales: Usar XML (como XHTML5) garantiza documentos perfectamente jerarquizados y facilita la interoperabilidad con otros sistemas basados en XML [3], [12]. Sin embargo, HTML5 (texto/html) es más flexible y tolerante, por eso es la opción estándar para páginas web normales. XML (y sus lenguajes) es preferible cuando se necesita definir datos estructurados o documentos especializados (por ejemplo, RSS para noticias, SVG para gráficos). Es importante entender las diferencias de sintaxis y validar correctamente cada tipo de documento dependiendo de la versión (HTML5 o XHTML5).

2.2. CSS para el Diseño y la Presentación Web

CSS define cómo deben mostrarse los elementos HTML en distintos medios (pantallas, impresión, voz, etc.) [13]. Su sintaxis básica consiste en reglas con selectores y

declaraciones {propiedad: valor;} aplicadas a etiquetas, clases o IDs. El CSS permite aplicar estilos globales o específicos y se hereda siguiendo la cascada de estilos. El uso de hojas externas (<link rel="stylesheet">) es preferido para mantener separación de preocupaciones. En diseño web actual, CSS incluye unidades flexibles (% , em, rem, vh/vw) y sistemas de columnas o cajas que facilitan la maquetación adaptable [13], [14]. Las buenas prácticas indican usar selectores específicos cuando sea necesario, evitar el abuso de IDs (prefiriendo clases reutilizables) y resets/normalización CSS para base consistente en todos los navegadores [13], [14].

2.2.1. Fundamentos de CSS.

Los fundamentos de CSS incluyen la forma de definir reglas y aplicar estilos. Una regla CSS típica:



```
1 p {  
2   color: blue;  
3   margin: 1rem;  
4 }
```

Fig. 9. Selector de etiqueta en CSS.

El selector p aplica color: blue y margin: 1em a todos los párrafos. Se pueden usar selectores de etiqueta, clase o #id [13].

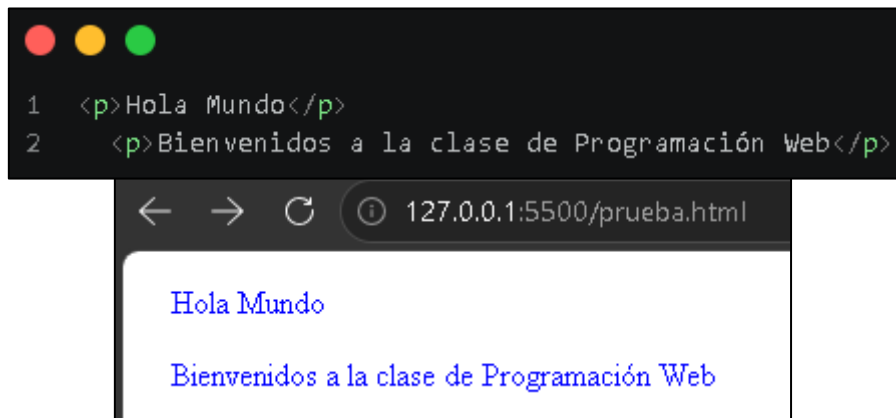


Fig. 10. Visualización de selector de etiqueta aplicado a HTML.

CSS funciona en cascada: las reglas posteriores o más específicas sobrescriben a las anteriores sí coinciden en el mismo elemento. El navegador además proporciona estilos por defecto (user-agent stylesheet), por lo que es común usar resets (p.ej., Normalize.css) para partir de un lienzo vacío [13]. Las propiedades CSS abarcan aspectos de presentación como tipografía (font-family, font-size), color (color, background-color), espaciados (margin, padding), bordes (border), dimensiones (width, height), visualización (display, position) y otras. Por ejemplo: Usar hoja de estilos externa y clases semánticas (.container) facilita la mantenibilidad [13].



Fig. 11. Uso de clases semánticas.

Ventajas: separación de contenido y presentación, reutilización de estilos y mejor accesibilidad.

Limitaciones: errores de cascada o especificidad pueden confundir, y estilos inline dificultan mantenimiento. Es buena práctica nombrar clases descriptivas, comentar reglas complejas, y validar el CSS (con <https://jigsaw.w3.org/css-validator>) para evitar sintaxis inválida que impida cargar el estilo [13].

2.2.2. Modelo de caja y maquetación CSS.

El modelo de caja CSS define cómo se calcula el tamaño de un elemento. Cada elemento se representa como una caja rectangular compuesta por: content-box (contenido), padding (relleno interno), border (borde) y margin (espacio externo) [14]. Por ejemplo:

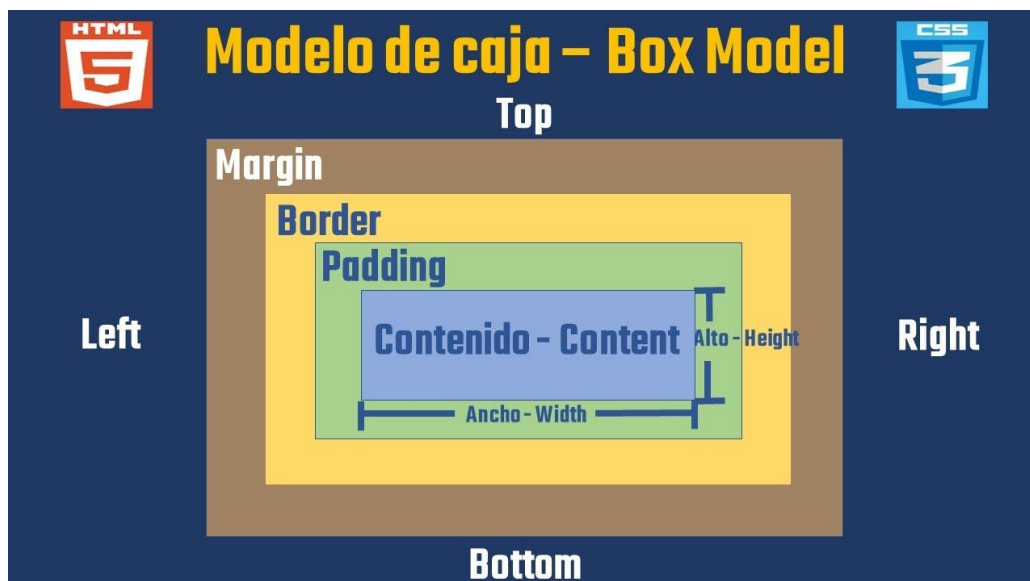


Fig. 12. Representación de modelo de caja.

En el modelo estándar, la caja ocupa 200px (contenido) + 210px (padding) + 25px (borde) = 230px de ancho total (el margen se añade después de la caja). En el modelo alternativo (box-sizing: border-box), el ancho de 200px ya incluye padding y borde, simplificando los cálculos [14]. Se recomienda aplicar box-sizing: border-box globalmente para un control más intuitivo del tamaño [14]. El diseño básico usa valores de display: block (cajas de bloque apiladas) e inline (elementos en línea). El modelo de caja influye en la disposición: margin puede colapsar entre elementos vecinos verticales (un fenómeno a tener en cuenta) [14]. Ejemplo:

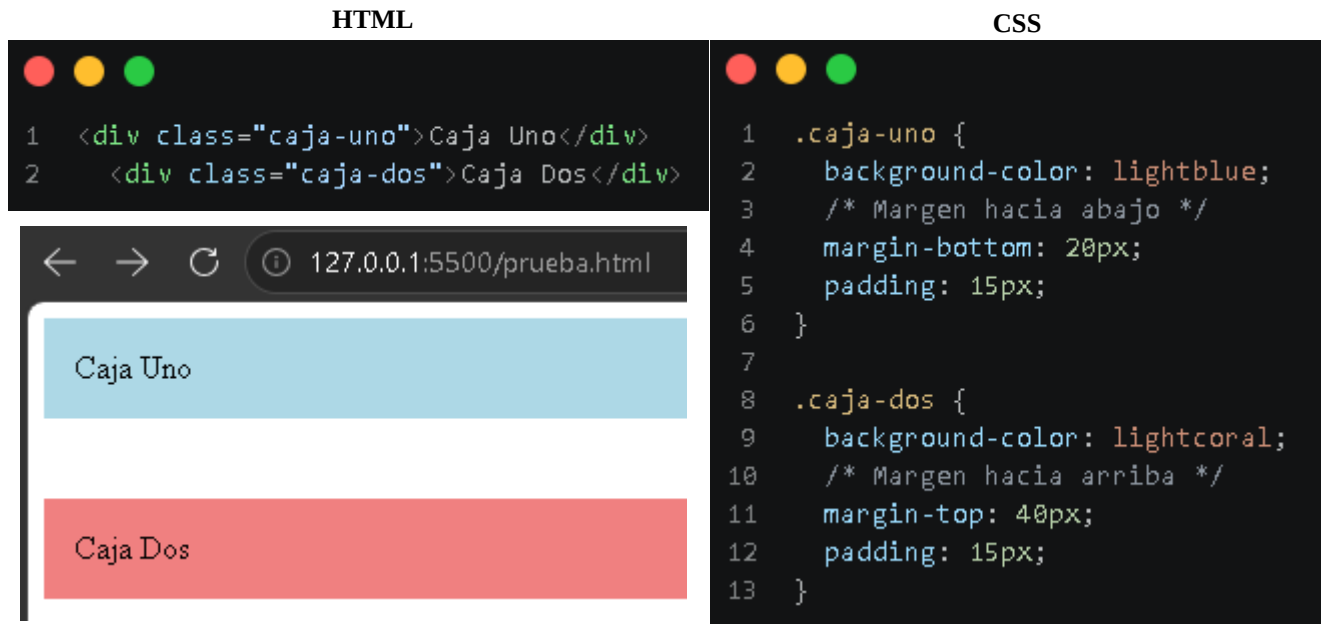


Fig. 13. Fenómeno de margin entre elementos verticales vecinos.

Notamos que la separación entre las dos cajas es de 40px (el margen de la segunda caja), ignorando completamente los 20px de la primera. No se acumulan.

Ventajas: entender el box model permite predecir el espaciado exacto de elementos.

Limitaciones: sin border-box puede complicar el tamaño. Buenas prácticas: usar abreviaturas (p.ej., margin: 10px 5px;), y CSS lógico (margin-block, margin-inline) para diseño bidireccional [13], [14].

2.2.3. Diseño Flexbox y CSS Grid.

Flexbox es un sistema de diseño unidimensional para alinear elementos en una fila o columna de forma flexible [15]. Se activa en el contenedor padre con (**display: flex;**). Por ejemplo:

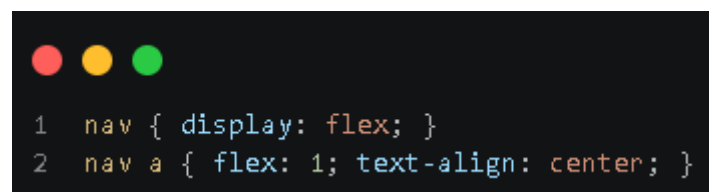


Fig. 14. Display flex en css.

Esto reparte equitativamente los enlaces (<a>) en el <nav> [15]. Flexbox facilita centrar vertical/horizontalmente (justify-content, align-items) y distribuir espacios dinámicamente [15]. Ejemplo:

```
1  /* Estilos del contenedor principal */
2  .contenedor-flex {
3      /* 1. Activamos Flexbox en el padre */
4      display: flex;
5
6      /* 2. Alineación horizontal: distribuye el espacio libre uniformemente entre las cajas */
7      justify-content: space-around;
8
9      /* 3. Alineación vertical: centra los elementos si el contenedor es más alto */
10     align-items: center;
11
12     /* Estilos visuales complementarios */
13     background-color: #f0f0f0;
14     height: 200px;
15     padding: 10px;
16     border: 2px solid #ccc;
17 }
18
19 /* Estilos para las cajas de adentro */
20 .caja {
21     background-color: #3498db;
22     color: white;
23     font-size: 24px;
24     font-family: sans-serif;
25     font-weight: bold;
26     width: 80px;
27     height: 80px;
28
29     /* Centramos el número dentro de cada caja usando también flexbox */
30     display: flex;
31     justify-content: center;
32     align-items: center;
33     border-radius: 8px;
34 }
```

Fig. 15. Código CSS para el uso de flexbox en las cajas

```
1  <!-- El contenedor padre (Flex Container) -->
2  <div class="contenedor-flex">
3      <!-- Los elementos hijos (Flex Items) -->
4      <p class="caja">Caja 1</p>
5      <input type="text" class="caja" value="Caja 2">
6      <button class="caja">Caja 3</button>
7  </div>
```

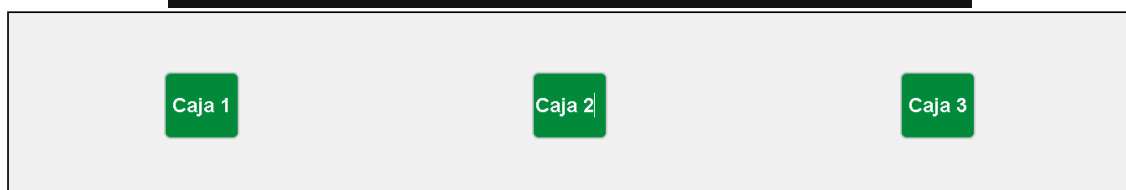


Fig. 16. Código HTML y visualización de cajas usando flexbox

Ventajas: solución a centrado vertical, reparto igual de espacio, ordenamiento fácil (order). **Limitaciones:** no maneja filas y columnas simultáneamente (solo una dimensión).

CSS Grid Layout es bidimensional: define filas y columnas. En el contenedor se usa `display: grid;` y se definen pistas con `grid-template-columns/rows` [16]. Por ejemplo:

```
1 .contenedor-grid {
2   /* 1. Activamos la rejilla */
3   display: grid;
4
5   /* 2. Creamos 2 columnas: la primera toma 2 partes del espacio y la segunda 1 parte */
6   grid-template-columns: 2fr 1fr;
7
8   /* 3. Añadimos el espacio de separación (gap) entre ellas */
9   gap: 20px;
10
11   background-color: #f0f0f0;
12   padding: 15px;
13 }
14
15 /* Estilos visuales para las columnas */
16 .columna-grande, .columna-pequeña {
17   background-color: #2ecc71;
18   color: white;
19   padding: 20px;
20   font-family: sans-serif;
21   font-weight: bold;
22   border-radius: 6px;
23 }
```

Fig. 17. Código CSS aplicando Grid a HTML.

Crea 2 columnas (2:1 ancho) con espacio (gap) entre ellas [16]. Se pueden posicionar elementos en áreas específicas (grid-area) o por línea de cuadrícula.

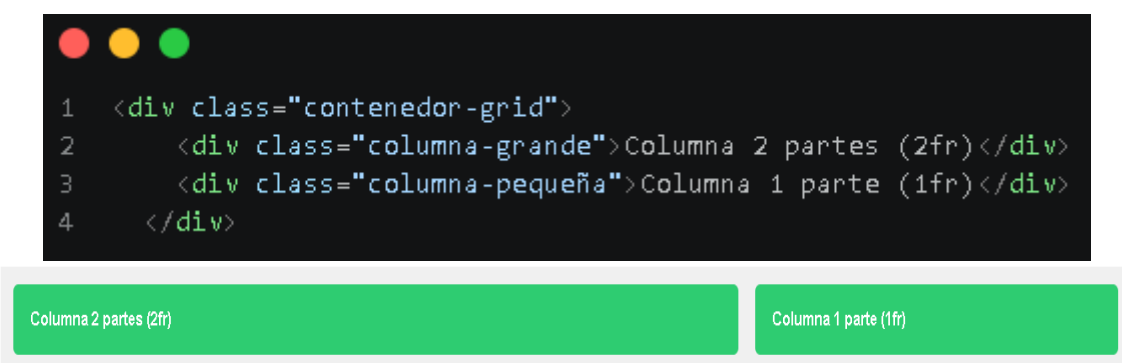


Fig. 18. Código HTML y visualización de las columnas.

Ventajas: permite diseños complejos (masonry, dashboards) con pocas reglas. **Limitaciones:** mayor complejidad inicial, y cierto soporte parcial en navegadores muy antiguos. Buenas prácticas: combinar Grid para la estructura principal (diseño de página) y Flexbox para controles individuales.

Ambos modelos ofrecen alternativas modernas al antiguo uso de float o position: absolute [15], [16]. Por ejemplo, para tres columnas iguales con Flexbox:

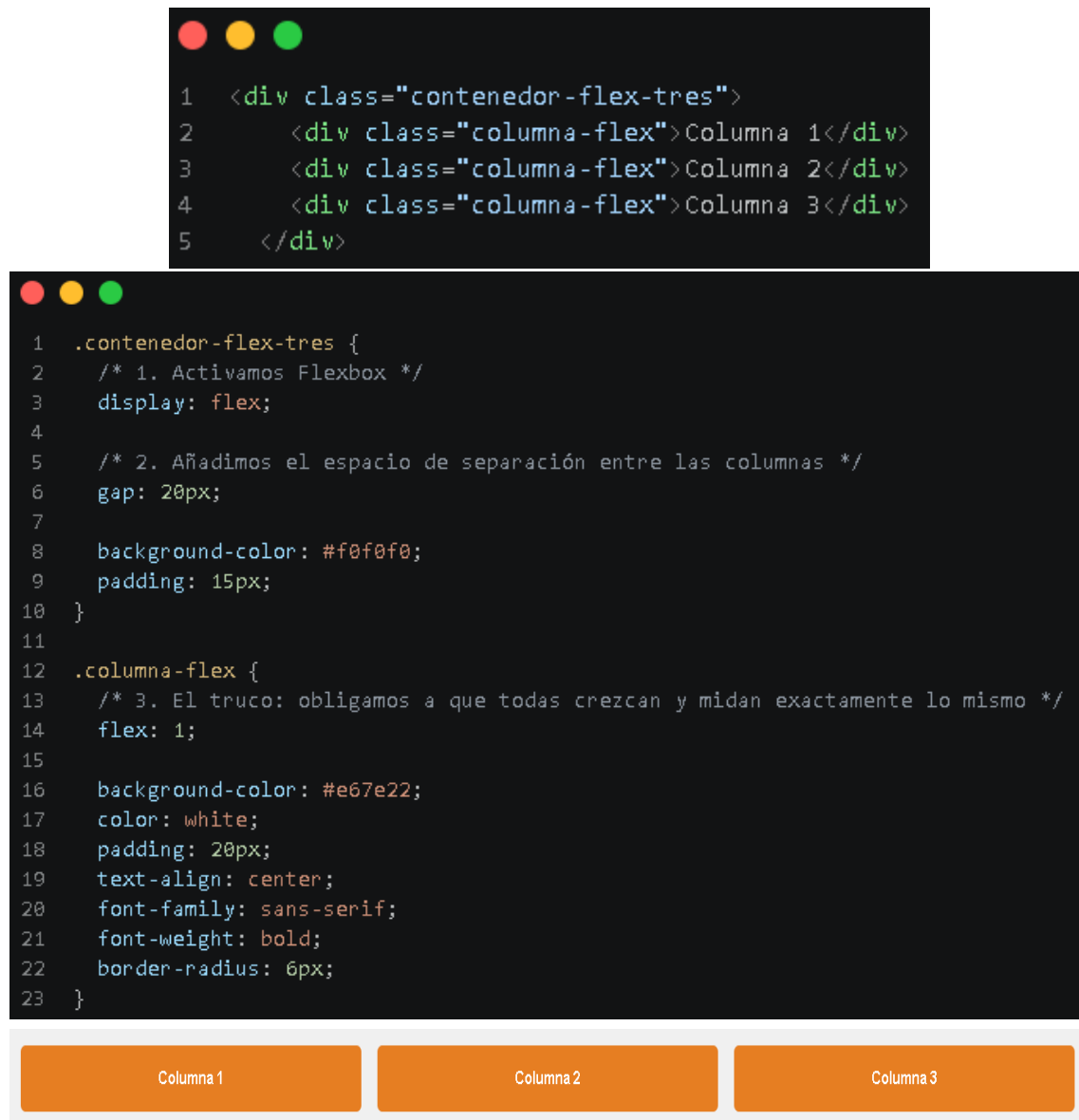


Fig. 19. Uso de flexbox para 3 columnas iguales.

Y con grid:



Fig. 20. Uso de Grid para 3 columnas iguales.

2.2.4. Diseño responsivo y mobile first.

El diseño responsivo adapta la web a cualquier dispositivo. Emplea media queries (@media) para cambiar estilos según ancho de pantalla, resolución o capacidad del dispositivo [17]. El enfoque mobile-first prioriza estilos móviles (pequeñas pantallas) y escala hacia tamaños mayores: por ejemplo, primero definir reglas base para max-width, y luego @media (min-width: 600px) para pantallas más grandes [17]. Ejemplo:

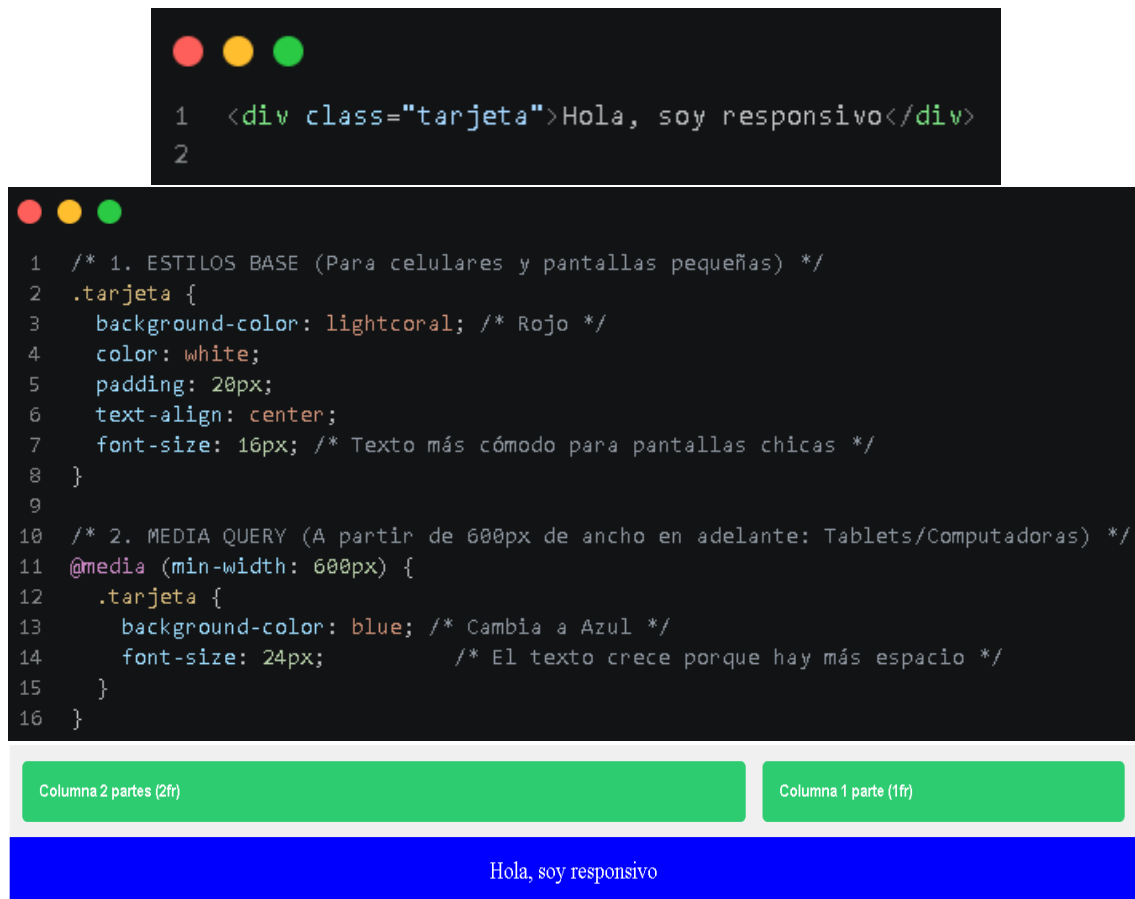


Fig. 21. Ejemplo de Mobile-First.

También se ajustan fuentes (font-size en rem o vw), imágenes (max-width: 100%), y se usa `<meta name="viewport" content="width=device-width,initial-scale=1.0">` en HTML para que en móviles el ancho físico se mapee correctamente [17].

Ventajas: mejora la experiencia en móviles, SEO y rendimiento; **limitaciones:** requiere planificación extra de layout. Buenas prácticas: diseñar la interfaz primero para móvil, probar con herramientas de navegador (responsive mode), y proporcionar imágenes adaptativas (`<picture>` o `srcset`) [17]. Frameworks modernos (Bootstrap, etc.) siguen esta filosofía. Además, combinaciones de Grid/Flexbox y media queries permiten reorganizar elementos (p.ej., de una columna en móvil a varias en escritorio) [17].

2.2.5. CSS3 avanzado y preprocesadores.

CSS3 introdujo funcionalidades avanzadas: transiciones y animaciones permiten animar cambios de estado sin JavaScript (p.ej., `transition: background 0.3s;`) [18]. Las transformaciones 2D/3D (`transform: rotate()`, `scale()`) y filtros (`filter: blur()`, etc.) enriquecen la interfaz. Otra característica clave son las propiedades personalizadas (`--mi-color: #c0ffee; color: var(--mi-color);`), que actúan como variables CSS y facilitan temas dinámicos [19]. Además, CSS soporta pseudoclasas avanzadas (`:nth-child`, `:not()`) y layouts inusuales (multi-column, clip-path, blend-mode).

Para código escalable, se usan preprocesadores CSS (Sass, Less, Stylus). Estos añaden sintaxis extendida (variables, anidamiento de selectores, mixins) que luego se compilan a CSS estándar. Por ejemplo, Sass permite:



```
1 $main-color: #09c;  
2 .nav {  
3   a { color: $main-color; &:hover { color: darken($main-color, 10%); } }  
4 }  
5
```

Fig. 22. Ejemplo de código SCSS.

y produce el CSS equivalente sin variables.



```
1 .nav a {  
2   color: #09c;  
3 }  
4  
5 .nav a:hover {  
6   color: #0077aa; /* Este es el azul original pero un 10% más oscuro */  
7 }
```

Fig. 23. Ejemplo del código SCSS a CSS.

Ventajas: código más modular, reutilizable y mantenible.

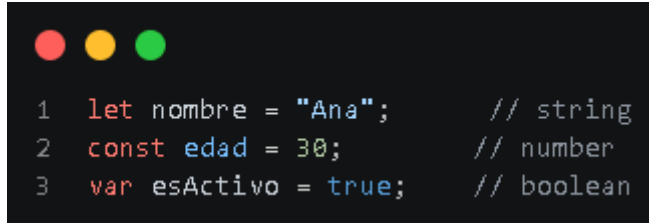
Limitaciones: paso adicional de compilación y complejidad en la configuración. Buenas prácticas incluyen evitar selectores demasiado específicos, comentar mixins complejos y minimizar dependencias de frameworks externos innecesarios. En proyectos actuales, muchos equipos combinan CSS3 moderno con preprocesadores para un flujo de trabajo eficiente.

2.3. JavaScript para Aplicaciones Web.

JavaScript es un lenguaje de programación interpretado que complementa a HTML y CSS para crear páginas web dinámicas e interactivas[20]. En un navegador, JavaScript accede al DOM (Modelo de Objetos del Documento) y a APIs web para modificar la página tras la carga inicial. Según MDN, “JavaScript añade interactividad a una página web y permite crear aplicaciones web ricas” [2]. Esto incluye responder a acciones del usuario (clics, teclado), actualizar información sin recargar (AJAX/fetch), animar elementos, almacenar datos locales (LocalStorage) y más. Ventajas de JavaScript: es omnipresente en navegadores, tiene amplias bibliotecas y frameworks (React, Angular, Vue, etc.) y permite escribir desde pequeños scripts a aplicaciones SPA complejas. Limitaciones: la calidad varía con el navegador (aunque hoy casi todos soportan ES6+), y es menos adecuado para cálculo pesado (sin Web Workers). Buenas prácticas: mantener separación de lógica (JS) y presentación (CSS/HTML), usar 'strict mode' para evitar errores silenciosos, modularizar el código (módulos ES6) y probar con herramientas (eslint, debuggers) [2], [20].

2.3.1. Fundamentos de JavaScript.

JavaScript es débilmente tipado y maneja tipos básicos como Number, String, Boolean, Object, Array, null y undefined [21]. Las variables se declaran con let, const o var (aunque var es hoistada y global, por lo que ahora se recomiendan let y const para alcance de bloque) [21]. Ejemplo:



```
1  let nombre = "Ana";      // string
2  const edad = 30;         // number
3  var esActivo = true;     // boolean
```

Fig. 24. Variables básicas de JavaScript.

El uso de const define constantes inmutables, mientras que let permite reasignar el valor. JavaScript evalúa expresiones aritméticas y concatenaciones. Por ejemplo, **console.log("Hola, " + nombre)** produce Hola, Ana. Se pueden usar operadores como +, -, *, /, %, y operadores lógicos (&&, ||, !). También existen literales de plantilla (template literals) con comillas invertidas `Hola, \${nombre}` para facilitar inserciones.

Ventajas: sintaxis flexible, sin compilación previa, orientado a objetos basada en prototipos.

Limitaciones: tipado dinámico puede causar errores en tiempo de ejecución (p. ej., "5" + 2 produce "52").

Buenas prácticas: declarar siempre variables con let/const (no var), usar nombres claros, y emplear el modo estricto "use strict" para detectar errores comunes.

2.3.2. Estructuras de control y funciones.

JavaScript dispone de estructuras de control clásicas: if...else (condicionales), switch (selección múltiple), bucles for, while, do...while y métodos de iteración (array.forEach(), etc.) [22]. Ejemplo de if:



```
1  if (edad >= 18) {
2    console.log("Adulto");
3  } else {
4    console.log("Menor de edad");
5  }
6
```

Fig. 25. Estructura de condicionales (if... else).

Y de bucle for:



```
1 for (let i = 0; i < 5; i++) {
2   console.log(i);
3 }
```

Fig. 26. Estructura de bucle for.

Las funciones pueden declararse con `function nombre(...) {...}` o como funciones flecha (`const suma = (a, b) => a + b;`). Por ejemplo



```
1 function saludar(nombre) {
2   return `Hola, ${nombre}!`;
3 }
4 console.log(saludar("Luisa")); // "Hola, Luisa!"
5
```

Fig. 27. Funcion para saludar.

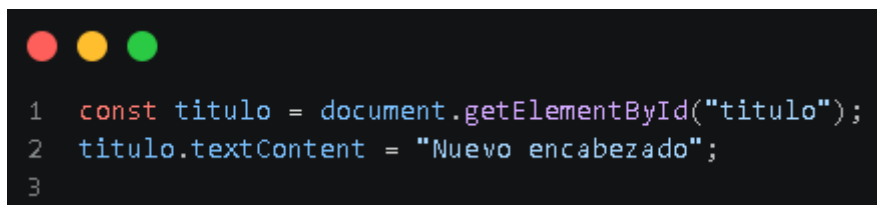
Las funciones son de primer nivel y pueden pasarse como argumentos o asignarse a variables.

Ventajas: las estructuras de control y funciones permiten lógica compleja y modularidad. Limitaciones: el alcance (scope) confuso de `var` y la gestión de contextos (`this`) en **funciones** convencionales pueden generar errores.

Buenas prácticas: usar `===` en lugar de `=` (evita coerción), emplear bloques `{}` en condicionales para claridad, y definir funciones reutilizables. También se recomienda manejo de errores con `try...catch` para funciones que puedan fallar.

2.3.3. Manipulación del DOM y eventos.

El DOM (Document Object Model) es la representación jerárquica del HTML de una página en el navegador. JavaScript accede al DOM para leer o modificar la estructura de la página en tiempo real [23]. Ejemplos comunes:

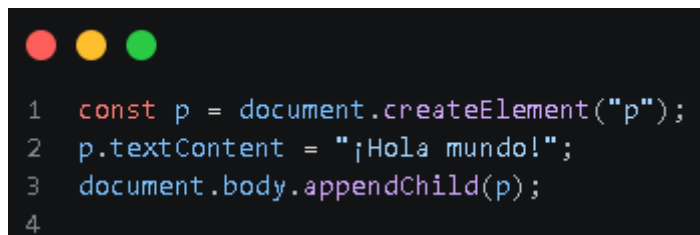


```
1 const titulo = document.getElementById("titulo");
2 titulo.textContent = "Nuevo encabezado";
3
```

Fig. 28. Código de JavaScript para cambiar el texto de un elemento en HTML.

Esto obtiene el elemento con `id="titulo"` y cambia su texto.

Para crear o eliminar nodos:



```
1 const p = document.createElement("p");
2 p.textContent = "¡Hola mundo!";
3 document.body.appendChild(p);
4
```

Fig. 28. Código de JavaScript para insertar nuevo texto en HTML.

Aquí se añade un nuevo párrafo al final del <body>. JavaScript también puede cambiar estilos (`element.style.color = "red"`), clases (`element.classList.add("activo")`), atributos, etc.

Los eventos son señales del navegador (clics, teclas, carga, etc.) que podemos manejar con `addEventListener` [23]. Por ejemplo:



```
1 const btn = document.querySelector("button");
2 btn.addEventListener("click", () => {
3   alert("¡Botón clickeado!");
4 });
```

Fig. 29. Código de JavaScript para ejecutar un evento.

En este ejemplo, al hacer clic en el botón se ejecuta la función flecha y aparece un mensaje. El objeto evento (event) provee detalles (como `event.target`). Es importante usar `event.preventDefault()` para cancelar comportamientos por defecto (como al enviar un formulario).

Ventajas: el manejo de DOM y eventos permite enriquecer las páginas con interactividad inmediata.

Limitaciones: manipulaciones excesivas pueden degradar rendimiento; por ello, se aconseja el event delegation (añadir oyentes a contenedores) y limitar repintados.

Buenas prácticas: separar código de control de eventos de la estructura HTML, limpiar manejadores cuando no son necesarios (`removeEventListener`), y evitar `innerHTML` sin validar para prevenir XSS.

2.3.4. JavaScript moderno (ES6+) y programación asíncrona.

Desde ES6 (2015) en adelante, JavaScript incorporó nuevas características: clases, módulos, template literals, destructuring, y más. Por ejemplo, una clase sencilla:

```
1  class Persona {
2    constructor(nombre) {
3      this.nombre = nombre;
4    }
5
6    saludar() {
7      return `Hola, soy ${this.nombre}`;
8    }
9  }
10
```

Fig. 30. Clase Persona en Javascript.

También se usan import/export para modularizar código. Otras mejoras incluyen arrow functions, spread/rest operators y promesas integradas. En programación asíncrona, JavaScript usa promesas y las funciones async/await para gestionar tareas que toman tiempo (peticiones HTTP, temporizadores) de forma secuencial. Por ejemplo:

```
1  async function obtenerDatos() {
2    try {
3      const respuesta = await fetch("/api/datos");
4      const datos = await respuesta.json();
5      console.log(datos);
6    } catch (e) {
7      console.error("Error:", e);
8    }
9  }
```

Fig. 31. Funcion asíncrona para obtener datos.

Este código usa await para esperar la respuesta de fetch() sin bloquear el hilo principal [23], [24].

Ventajas: sintaxis más limpia y legible que callbacks, y manejo de errores integrado.

Limitaciones: compatibilidad (aunque hoy los navegadores modernos soportan ES2023), y aún requiere manejo adecuado de errores.

Buenas prácticas: usar siempre async/await dentro de try...catch, evitar mezclar callbacks con promesas, y optar por promesas nativas en lugar de bibliotecas externas (salvo casos especiales). El mundo moderno de JS también incluye el uso de herramientas de bundling (Webpack, Babel) para soportar ESNext y frameworks modernos (React/Vue/Angular).

Las métricas de rendimiento obtenidas mediante Lighthouse nos permitieron evaluar la velocidad y estabilidad de carga de la aplicación web. El First Contentful Paint (FCP) de 1.0 segundos indica el tiempo que tarda en mostrarse el primer contenido visible al usuario, mientras que el Largest Contentful Paint (LCP) de 1.0 segundos mide cuándo se renderiza el elemento principal de la página. El Total Blocking Time (TBT) de 270 ms representa el tiempo durante el cual el navegador estuvo bloqueado ejecutando tareas que pudieron afectar la interacción del usuario. Por otro lado, el Cumulative Layout Shift (CLS) obtuvo un valor de 0, lo que significa que no se produjeron movimientos inesperados de los elementos durante la carga. Finalmente, el Speed Index de 1.1 segundos refleja la rapidez con la que el contenido visible se muestra completamente al usuario.

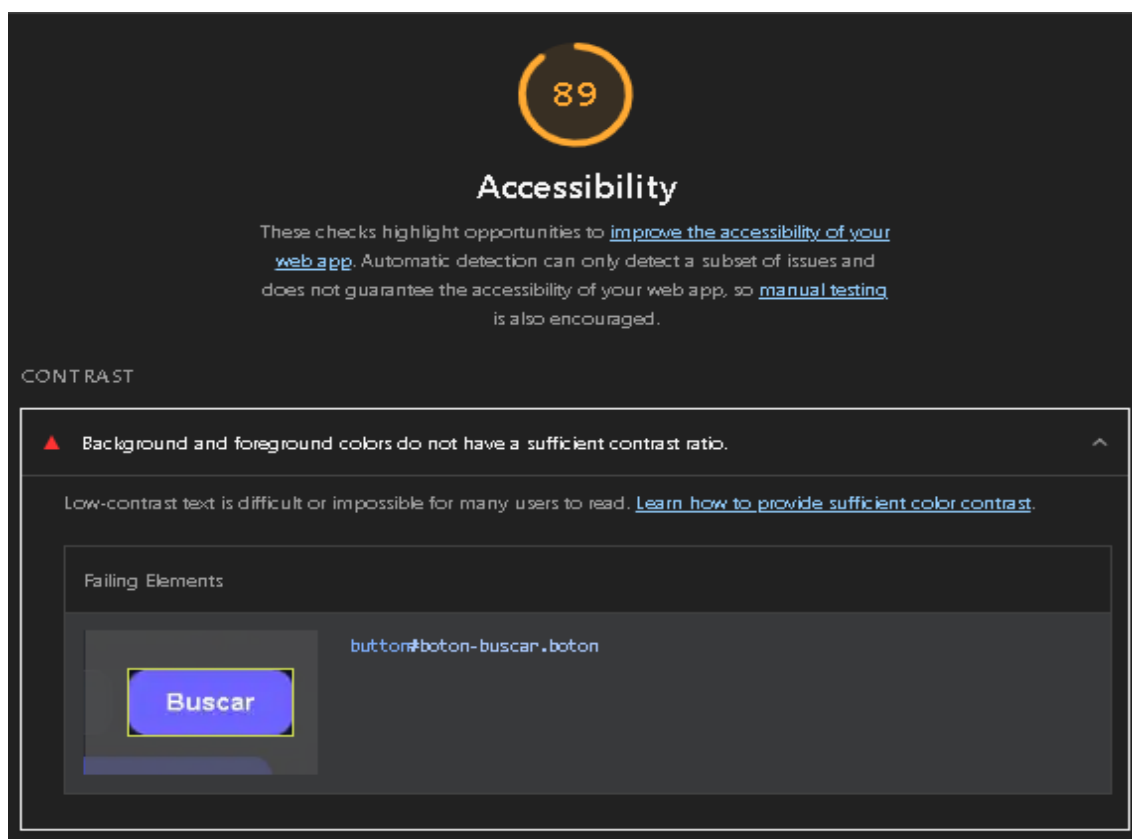


Fig. 34. Puntuación de accesibilidad.

La evaluación inicial de accesibilidad realizada con Lighthouse obtuvo una puntuación de 89/100, identificamos un problema relacionado con el contraste insuficiente entre el texto y el fondo en el botón de búsqueda. Este tipo de inconveniente dificulta la lectura para usuarios con discapacidad visual o baja percepción de contraste, ya que el texto no se distingue adecuadamente del color de fondo.

3.1.1. Corrección y mejora

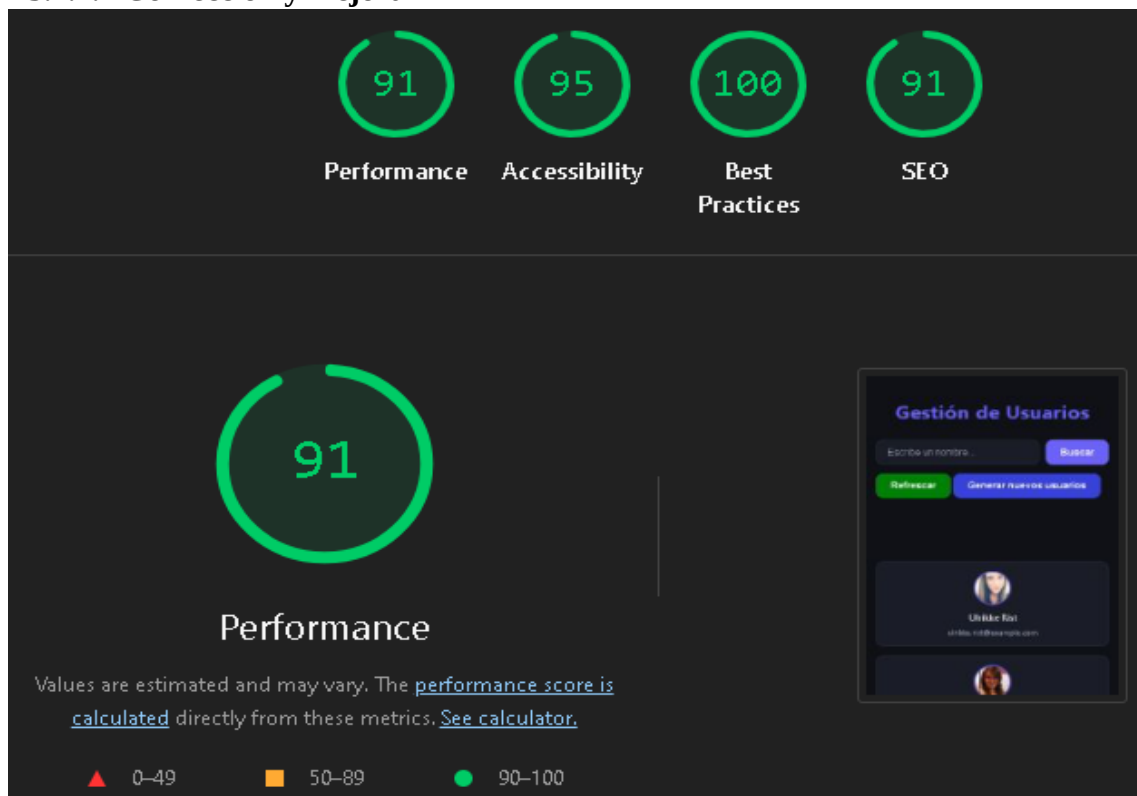


Fig. 35. Métricas de lighthouse (Mejoras)

Correcciones realizadas en el HTML: Se mejoró la estructura semántica de la página reemplazando el contenedor principal por la etiqueta `<main>`, permitiendo que los lectores de pantalla identifiquen correctamente el contenido principal del sitio. Además, se añadió una etiqueta `<label>` asociada al campo de búsqueda para proporcionar una descripción accesible del propósito del input, ya que los placeholders no sustituyen adecuadamente a las etiquetas. También se incorporó el atributo `aria-live="polite"` en el mensaje de estado para que los cambios dinámicos sean anunciados automáticamente por los lectores de pantalla. Finalmente, se utilizaron elementos `<section>` con atributos `aria-label` para describir las áreas donde se muestran las tarjetas y listas de usuarios, facilitando la navegación para usuarios que utilizan tecnologías de asistencia.

Correcciones realizadas en el CSS: Se mejoró el contraste visual aumentando la luminosidad del color utilizado en textos secundarios (`--color-texto-suave`), lo que incrementa la legibilidad sobre el fondo oscuro y ayuda a cumplir las recomendaciones de accesibilidad relacionadas con el contraste. Asimismo, se añadieron estilos de enfoque (`:focus`) para los campos de texto y botones mediante un contorno visible, permitiendo que los usuarios que navegan con teclado puedan identificar claramente qué elemento está seleccionado en cada momento. Finalmente, se incorporó la clase `.sr-only`, utilizada para ocultar visualmente elementos como etiquetas descriptivas sin eliminarlas de los lectores de pantalla, mejorando la accesibilidad sin afectar el diseño visual de la interfaz.

3.2. Auditoria con wave

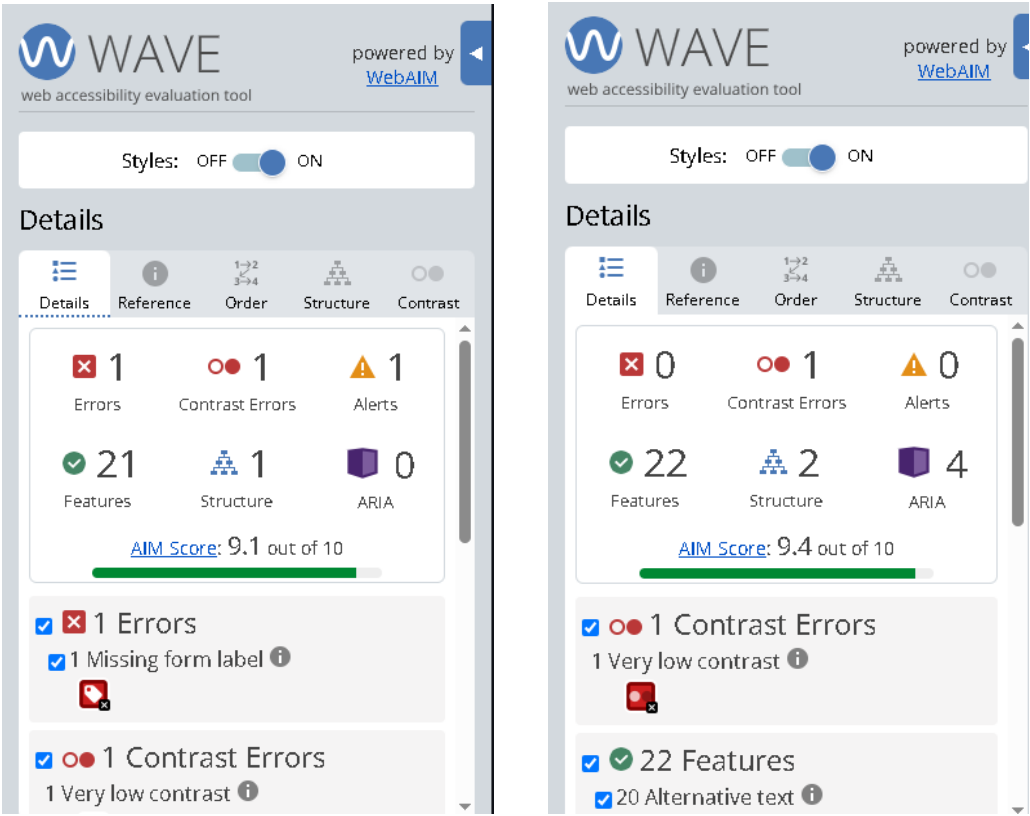


Fig. 36. Métricas de wave (Antes y después).

Error detectado por WAVE	Elemento afectado	Corrección aplicada
Missing form label	Campo de búsqueda (input)	Se agregó una etiqueta <label> asociada al campo de búsqueda para proporcionar una descripción accesible y eliminar el error detectado por WAVE.
Very low contrast	Textos secundarios y placeholders de la interfaz	Se ajustaron los colores utilizados en los textos de apoyo para incrementar el contraste respecto al fondo oscuro y mejorar la legibilidad.

Tabla 1. Detección y corrección de errores.

3.3. Tabla comparativa de métricas

Métrica	Antes	Después	Objetivo
Lighthouse Accessibility	89	95	≥ 90
Lighthouse Performance	95	91	≥ 70
Lighthouse Best Practices	100	100	≥ 90
Lighthouse SEO	91	91	≥ 90
WAVE Errors	1	0	0
WAVE Alerts	1	0	< 5

Errores W3C HTML	No evaluado	No evaluado	0
Errores W3C CSS	No evaluado	No evaluado	0
Ratio contraste mínimo	No cumple completamente	En revisión / Mejorado	≥ 4.5 : 1

4. Conclusión

Y finalizando, el desarrollo web moderno integra diferentes tecnologías que trabajan de manera conjunta para crear aplicaciones funcionales, interactivas y visualmente atractivas. HTML y HTML5 nos da la estructura y organización del contenido, CSS nos permite diseñar interfaces adaptables y responsivas, mientras que JavaScript añade dinamismo e interacción a las páginas web. El estudio de estos elementos y sus herramientas complementarias permite comprender las bases fundamentales para el desarrollo de aplicaciones web actuales, facilitando la creación de soluciones eficientes, accesibles y orientadas a las necesidades de los usuarios y del entorno tecnológico actual. Además, la utilización de herramientas de evaluación como Lighthouse y WAVE nos permitió analizar aspectos clave relacionados con el rendimiento, la accesibilidad, las buenas prácticas y la optimización para motores de búsqueda, identificando oportunidades de mejora y aplicando correcciones específicas. Gracias a ello, se logró incrementar la puntuación de accesibilidad, eliminar errores detectados en formularios y fortalecer la experiencia de usuario mediante una mejor estructura semántica, mayor contraste visual y soporte para tecnologías de asistencia, demostrando la importancia de incorporar criterios de calidad y accesibilidad durante todo el proceso de desarrollo web.

5. Referencias

- [1] Equipo editorial de IONOS, “Nociones básicas sobre HTML5,” IONOS. Accessed: May 12, 2026. [Online]. Available: <https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/que-es-html5/>
- [2] MDN Web Docs, “Resumen de tecnologías JavaScript,” developer.mozilla.org. Accessed: May 13, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/JavaScript_technologies_overview#:~:text=Whereas%20HTML%20%20defines%20a,and%20creates%20rich%20web%20a pplications
- [3] Whatwg Group community, “HTML,” Whatwg.org. Accessed: May 12, 2026. [Online]. Available: <https://html.spec.whatwg.org/multipage/>
- [4] DraftECMA, “ECMAScript Lenguaje de especificacion,” ECMAScript. Accessed: May 13, 2026. [Online]. Available: <https://tc39.es/ecma262/>
- [5] Mozilla Developer Network, “HTML: Lenguaje de Marcado de Hipertexto,” developer.mozilla.org. Accessed: May 12, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML>
- [6] W3C, “Un vocabulario y APIs asociadas para HTML y XHTML,” W3C.org. Accessed: May 12, 2026. [Online]. Available:

- <https://www.w3.org/TR/2018/SPSD-html5-20180327/introduction.html#html-vs-xhtml#:~:text=The%20first%20such%20concrete%20syntax,HTML%205>
- [7] Vlado Pavlik, “HTML Semántico: Qué Es y Cómo Usarlo Correctamente,” SEMRUSH. Accessed: May 19, 2026. [Online]. Available: <https://es.semrush.com/blog/html-semantic/>
- [8] freecodecamp.org, “Elementos semánticos HTML5 explicados,” freeCodeCamp. Accessed: May 12, 2026. [Online]. Available: <https://www.freecodecamp.org/espanol/news/elementos-semanticos-html5-explicados/#:~:text=Los%20elementos%20sem%C3%A1nticos%20HTML%20s on,legible%20por%20humanos%20y%20m%C3%A1quinas>
- [9] Mozilla Developer Network, “Uso de la validación de formularios HTML y la API de Validación de Restricciones,” developer.mozilla.org. Accessed: May 12, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Constraint_validation#:~:text=The%20creation%20of%20web%20forms,using%20the%20Constraint%20Validation%20API
- [10] Mozilla Developer Networ, “Tipos de input de HTML5,” developer.mozilla.org. Accessed: May 12, 2026. [Online]. Available: https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Forms/HTML5_input_types#:~:text=Cuando%20se%20utiliza%20este%20valor,la%20 siguiente%20captura%20de%20pantalla
- [11] Mozilla Developer Network, “Introduccion para desarrollo de juegos para la Web,” developer.mozilla.org. Accessed: May 12, 2026. [Online]. Available: <https://developer.mozilla.org/es/docs/Games/Introduction#:~:text=Juntas%2C%20estas%20dos%20tecnolog%C3%ADas%20le,forma%20de%20hacer%20gr%C3%A1ficos%202D>
- [12] Mozilla Developer Network, “Introducción a XML,” developer.mozilla.org. Accessed: May 12, 2026. [Online]. Available: https://developer.mozilla.org/es/docs/Web/XML/Guides/XML_introduction#:~:text=XML%20es%20un%20lenguaje%20de,de%20diferentes%20sistemas%2C%20como%20Internet
- [13] MDN Web Docs, “CSS Fundamentos,” developer.mozilla.org. Accessed: May 13, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/curriculum/core/css-fundamentals/#:~:text=CSS%20enables%20you%20to%20add,to%20understand%20how%20CSS%20works>
- [14] MDN Web Docs, “El modelo de caja,” developer.mozilla.org. Accessed: May 13, 2026. [Online]. Available: https://developer.mozilla.org/es/docs/Learn_web_development/Core/Styling_basics/Box_model#:~:text=,es%20la%20capa%20m%C3%A1s%20externa
- [15] MDN Web Docs, “Flexbox,” developer.mozilla.org. Accessed: May 13, 2026. [Online]. Available: https://developer.mozilla.org/es/docs/Learn_web_development/Core/CSS_layout

- /Flexbox#:~:text=Flexbox%20es%20un%20m%C3%A9todo%20de,expondremo
s%20todas%20sus%20caracter%C3%ADsticas%20b%C3%A1sicas
- [16] MDN Web Docs, “Cuadrículas,” developer.mozilla.org. Accessed: May 13, 2026. [Online]. Available: https://developer.mozilla.org/es/docs/Learn_web_development/Core/CSS_layout/Grids#:~:text=css
- [17] MDN Web Docs, “Diseño web responsivo,” developer.mozilla.org. Accessed: May 13, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design#:~:text=,display%20appropriately%20on%20mobile%20devices
- [18] MDN Web Docs, “Transiciones de CSS,” developer.mozilla.org. Accessed: May 13, 2026. [Online]. Available: <https://developer.mozilla.org/es/docs/Web/CSS/Guides/Transitions/Using#:~:text=Las%20transiciones%20CSS%2C%20parte%20del,de%20las%20propiedades%20CSS%2C>
- [19] MDN Web Docs, “Uso de propiedades personalizadas (variables) en CSS,” developer.mozilla.org. Accessed: May 13, 2026. [Online]. Available: https://developer.mozilla.org/es/docs/Web/CSS/Guides/Cascading_variables/Using_custom_properties#:~:text=En%20CSS%2C%20las%20propiedades%20personalizadas,color
- [20] Erick, “¿Qué es JavaScript? El código de JavaScript en español sencillo,” freeCodeCamp. Accessed: May 13, 2026. [Online]. Available: <https://www.freecodecamp.org/espanol/news/que-es-javascript-el-codigo-de-javascript-en-espanol-sencillo/#:~:text=JavaScript%20es%20usado%20con%20HTML,de%20construcci%C3%B3n%20del%20desarrollo%20web>
- [21] Sarah Chima Atuonwu, “Var, Let y Const – ¿cuál es la diferencia?,” freeCodeCamp. Accessed: May 13, 2026. [Online]. Available: <https://www.freecodecamp.org/news/var-let-and-const-whats-the-difference/#:~:text=One%20of%20the%20features%20that,this%20article%20is%20for%20you>
- [22] MDN Web Docs, “if...else,” developer.mozilla.org. Accessed: May 13, 2026. [Online]. Available: <https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/if...else#:~:text=La%20sentencia%20,ser%C3%A1%20ejecutada>
- [23] MDN Web Docs, “Fundamentos de JavaScript,” developer.mozilla.org. Accessed: May 13, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/curriculum/core/javascript-fundamentals/#:~:text=,do%20asynchronous%20programming%20in%20JavaScript>
- [24] MDN Web Docs, “Introducción a JavaScript asíncrono,” developer.mozilla.org. Accessed: May 13, 2026. [Online]. Available:

https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Async_JS/Introducing#:~:text=En%20este%20art%C3%ADculo%2C%20explicaremos%20qu%C3%A9,han%20implementado%20hist%C3%B3ricamente%20en%20JavaScript

6. Anexo

Interfaz de la pagina

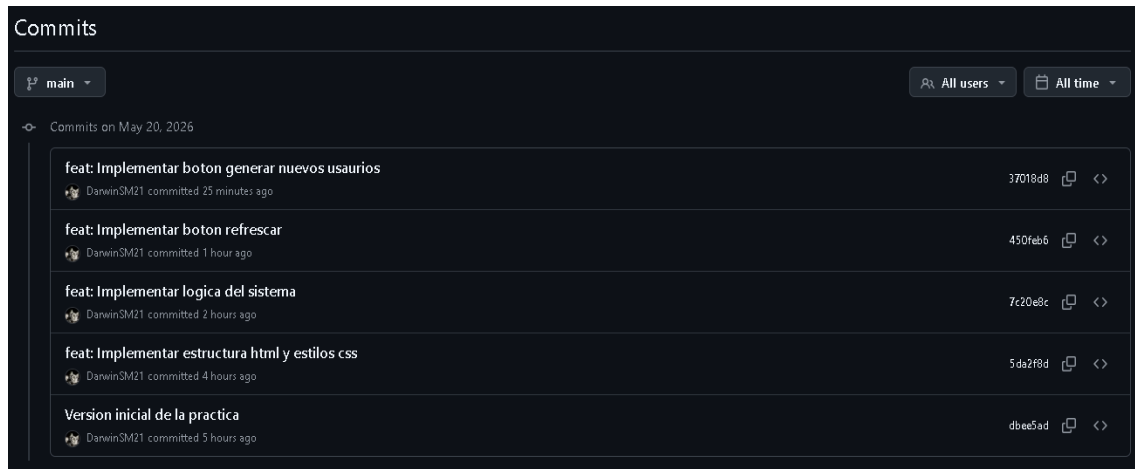


Resultado de búsqueda en la pagina



Commits realizados

Commits versión 1



A screenshot of the GitHub 'Commits' page for the 'main' branch. The page shows a list of five commits by user DarwinSM21. The commits are ordered from newest to oldest. Each commit entry includes the commit message, the user's profile picture and name, the time since the commit was made, and the commit hash with icons to view the commit details or the diff.

Commit Message	User	Time	Hash
feat: Implementar boton generar nuevos usuarios	DarwinSM21	committed 25 minutes ago	37018d8
feat: Implementar boton refrescar	DarwinSM21	committed 1 hour ago	450feb6
feat: Implementar logica del sistema	DarwinSM21	committed 2 hours ago	7c20e8c
feat: Implementar estructura html y estilos css	DarwinSM21	committed 4 hours ago	5da2f8d
Version inicial de la practica	DarwinSM21	committed 5 hours ago	dbee5ad

Commits versión 2



A screenshot of the GitHub 'Commits' page for the 'main' branch, showing a list of commits. The commits are ordered from newest to oldest. The first commit is a fix for accessibility, followed by five feature commits. Each commit entry includes the commit message, the user's profile picture and name, the time since the commit was made, and the commit hash with icons to view the commit details or the diff.

Commit Message	User	Time	Hash
fix(Accesibilidad: añadir etiquetas a los formularios, mejorar contraste y foco de teclado)	DarwinSM21	committed now	91edbdb
feat: Implementar boton generar nuevos usuarios	DarwinSM21	committed 3 weeks ago	37018d8
feat: Implementar boton refrescar	DarwinSM21	committed 3 weeks ago	450feb6
feat: Implementar logica del sistema	DarwinSM21	committed 3 weeks ago	7c20e8c
feat: Implementar estructura html y estilos css	DarwinSM21	committed 3 weeks ago	5da2f8d
Version inicial de la practica	DarwinSM21	committed 3 weeks ago	dbee5ad

Link del repositorio

https://github.com/DarwinSM21/PE_1_APPWEB.git